# KNRT MARKETPLACE

Technical Whitepaper

Version 1.0 | Date: 2026-01-15

Author: Kaan Nurettin Ramazan
Contact: kaan@knrt.io | GitHub: kaan-knrt

**LEGAL NOTICE**

This document is provided for informational purposes only.
It does not constitute a contract, offer, or solicitation.
Technical specifications are subject to change without notice.
Implementation details may differ from the reference architecture.
No warranty is provided regarding accuracy or completeness.

**KEY FEATURES**

- Decentralized Marketplace
- Cryptographic Identity Management
- Multi-Signature Escrow Protocol
- Hybrid Reputation System
- Legal Framework Integration
- Dispute Resolution Mechanism
- Cross-Platform Implementation

Document Classification: Technical Specification
Document Type: Whitepaper
Intended Audience: Developers, Architects, Security Researchers

## TABLE OF CONTENTS

1. EXECUTIVE SUMMARY 2
2. INTRODUCTION TO KNRT MARKETPLACE 2
2.1 Platform Overview 2
2.2 Design Principles 2
3. TECHNICAL ARCHITECTURE 2
3.1 Zero-Layer 2 Network 2
3.2 Event-Driven Microservices 2
3.3 Data Ownership 2
4. CORE FEATURES & FUNCTIONALITY 2
4.1 NFT Creation & Minting 2
4.2 Marketplace Trading 2
4.3 Reputation System 2
5. DATA ARCHITECTURE 2
5.1 Hybrid Storage Model 2
5.2 Data Schemas 2
5.3 Cryptographic Integrity 2
6. SECURITY MODEL 2
6.1 Threat Model 2
6.2 Cryptographic Primitives 2
6.3 Key Management 2
7. ECONOMIC MODEL 2
7.1 Fee Structure 2
7.2 Escrow Economics 2
7.3 Incentive Alignment 2
8. LEGAL FRAMEWORK 2
8.1 Jurisdiction 2
8.2 Dispute Resolution 2
8.3 Evidence Preservation 2
9. IMPLEMENTATION 2
9.1 Technology Stack 2
9.2 Deployment Architecture 2
9.3 Monitoring 2
10. ROADMAP 2
10.1 Phase 1: Foundation (Q1 2026) 2
10.2 Phase 2: Ecosystem (Q2 2026) 2
10.3 Phase 3: Governance (Q3 2026) 2
10.4 Phase 4: Decentralization (Q4 2026) 2
11. CONCLUSION 2
11.1 Summary 2
11.2 Vision 2
11.3 Call to Action 2
12. APPENDIX 2
12.1 Glossary 2
12.2 References 2
12.3 Contact 2

## 1. EXECUTIVE SUMMARY

KNRT Marketplace implements a peer-to-peer NFT marketplace with cryptographic escrow and reputation tracking. The system combines Ethereum Layer 2 settlement, IPFS content addressing, and Ed25519 cryptographic identity.

**Key Innovations**

- Zero-Custody Escrow: Multi-signature escrow where neither party nor platform controls funds unilaterally
- Cryptographic Reputation: Ed25519-signed reputation records with Merkle tree aggregation
- Hybrid Storage Model: Critical data on-chain, bulk data on IPFS with integrity verification
- Event-Driven Architecture: Microservices communicating via Redis Streams
- Legal Framework Integration: Jurisdiction-aware dispute resolution with evidence preservation

**Target Audience**

- Developers building decentralized marketplaces
- Security researchers evaluating cryptographic implementations
- Legal professionals assessing jurisdiction-aware dispute resolution
- Enterprises considering blockchain-based commerce infrastructure

## 2. INTRODUCTION TO KNRT MARKETPLACE

### 2.1 Platform Overview

KNRT Marketplace is a decentralized marketplace for digital assets and services. The platform supports:

- Digital Assets: NFTs, tokenized documents, software licenses
- Services: Freelance work, consulting, custom development
- Transaction Types: Fixed price, auctions, escrow-protected trades

The platform operates without custody of user funds. Smart contracts enforce escrow conditions, IPFS stores metadata with content addressing, and cryptographic signatures provide identity verification.

### 2.2 Design Principles

**Trust Minimization**

Trust is distributed across:

- Smart contracts (Ethereum Layer 2)
- Cryptographic primitives (Ed25519, SHA-256)
- Decentralized storage (IPFS)
- Reputation aggregators (multiple independent operators)

No single entity controls all three.

**Cost Minimization**

Ethereum Layer 2 (Optimism) reduces transaction costs by approximately 90% compared to Ethereum mainnet. Typical transaction costs:

- Mint: $0.01–$0.05
- Trade: $0.02–$0.10
- Escrow: $0.05–$0.20

**Data Ownership**

- Identity: Ed25519 keypairs generated locally, never transmitted
- Reputation: Signed records exportable to any compatible system
- Content: IPFS addressing enables retrieval from any node

**Jurisdiction Awareness**

- Dispute resolution supports multiple legal frameworks
- Evidence preservation meets cross-jurisdiction requirements
- Platform does not assume single-jurisdiction operation

## 3. TECHNICAL ARCHITECTURE

### 3.1 Zero-Layer 2 Network

Smart contracts deploy on Optimism, an Ethereum Layer 2 network. Transactions inherit Ethereum's security while achieving approximately 90% cost reduction.

**Why Not Mainnet?**

At 30 gwei gas price and $3,000 ETH:

- Simple transfer: $0.30
- Contract interaction: $3.00–$15.00
- Complex escrow operation: $15.00–$50.00

Layer 2 reduces these to $0.01–$0.50 while maintaining Ethereum security guarantees.

**Why Not Other L2s?**

Optimism provides:

- Mature tooling (Hardhat, OpenZeppelin)
- Active development ecosystem
- Clear path to Ethereum mainnet if needed
- Compatibility with standard Ethereum wallets

### 3.2 Event-Driven Microservices

All backend services communicate via Redis Streams. Services subscribe to relevant event streams and maintain local state. Failure in one service does not cascade to others.

```
Blockchain Indexer → Redis Streams → Reputation Aggregator
                   → Redis Streams → Notification Service
                   → Redis Streams → Analytics Service
```

**Benefits**

- Decoupled deployment: Services can be updated independently
- Fault tolerance: Failed consumers do not affect producers
- Scalability: Add consumers without modifying producers
- Observability: Event streams provide audit trail

**Implementation**

- Runtime: Bun 1.0+ (TypeScript)
- Framework: Custom event handlers
- Persistence: Redis Streams with consumer groups

### 3.3 Data Ownership Model

| Data | Storage | Ownership |
|---|---|---|
| Identity | Local keystore | User controls private key |
| Reputation | Signed records | Exportable, portable |
| Content | IPFS | Content addressing |
| Transactions | Blockchain | Immutable record |

**Identity**

Ed25519 keypairs generated locally. Private key never transmitted. Signature verification proves ownership without revealing private key.

**Reputation**

Reputation records are signed by aggregators and can be exported and verified by any compatible system. Multiple aggregators prevent single-point manipulation.

**Content**

Content addressing (IPFS CIDs) ensures retrieval from any node. Platform cannot modify stored content.

**Transactions**

Blockchain provides immutable record of escrow events. Dispute resolution can reference this record.

## 4. CORE FEATURES & FUNCTIONALITY

### 4.1 NFT Creation & Minting

Users create NFTs by uploading metadata to IPFS and calling the marketplace contract. Supported standards: ERC-721 (single item), ERC-1155 (batch).

**Metadata Structure**

```
{
  "name": "Example Asset",
  "description": "Description",
  "image": "ipfs://Qm...",
  "attributes": [
    {"trait_type": "Type", "value": "Example"}
  ]
}
```

**Minting Flow**

1. User uploads metadata to IPFS, receives CID
2. User calls mint(to, tokenURI) on contract
3. Contract emits Minted(tokenId, to, tokenURI)
4. Indexer detects event, updates database

### 4.2 Marketplace Trading

**Fixed Price**

- Seller lists at specified price in USDC
- Buyer approves USDC spending
- Buyer calls buyFixed(listingId)
- Contract transfers NFT to buyer, USDC minus fee to seller

**Escrow**

- Buyer creates escrow with seller, amount, deadline
- Buyer funds escrow
- Seller delivers off-chain
- Buyer confirms receipt
- Contract releases funds

**Auctions**

- Seller creates auction with reserve price and duration
- Bidders place bids (USDC held in escrow)
- Anti-snipe: bids in final 5 minutes extend auction by 5 minutes
- Winner calls settleAuction() after auction ends
- Losing bids refunded

### 4.3 Reputation System

**Ed25519 Signature Verification**

```
// Reputation record structure
interface ReputationRecord {
  subject: PublicKey,
  score: number,
  timestamp: Unix,
  evidence: [TxHash],
  aggregator: PublicKey,
  signature: Ed25519Sig
}

// Verification
function verify(record, aggregatorPubkey):
  return ed25519.verify(
    record.signature,
    hash(record.subject || record.score || record.timestamp || record.evidence),
    aggregatorPubkey
  )
```

**Merkle Tree Aggregation**

Aggregators collect reputation records and publish Merkle roots periodically. Individual records can be verified against the root without downloading the full tree.

**Sybil Resistance**

- Minimum transaction count before reputation accrues
- Weight by counterparty reputation
- Time decay: recent transactions weight more
- Dispute penalties: losing disputes reduces score

## 5. DATA ARCHITECTURE

### 5.1 Hybrid Storage Model

| Data | Storage | Rationale |
|---|---|---|
| Escrow state | Blockchain | Must be immutable for trustless execution |
| Reputation roots | Blockchain | Anchor for verification |
| Reputation records | Off-chain | Too expensive for full storage |
| Metadata | IPFS | Content addressing ensures integrity |
| Images | IPFS | Bandwidth cost prohibitive on chain |
| Indexes | Database | Rebuilds from blockchain events |

**Principle**

Blockchain provides ordering and finality. Off-chain systems provide speed and storage. Cryptography connects them without requiring trust.

### 5.2 Data Schemas

**Escrow**

```
struct Escrow {
  address buyer;
  address seller;
  address token;
  uint256 amount;
  uint256 deadline;
  bytes32 metadataHash;
  EscrowStatus status;
}
```

**Listing**

```
struct Listing {
  address seller;
  address nftContract;
  uint256 tokenId;
  address paymentToken;
  uint256 price;
  uint256 expiry;
  ListingStatus status;
}
```

**Auction**

```
struct Auction {
  address seller;
  address nftContract;
  uint256 tokenId;
  address paymentToken;
  uint256 reserve;
  uint256 startTime;
  uint256 endTime;
  address highBidder;
  uint256 highBid;
  bool settled;
}
```

### 5.3 Cryptographic Integrity

**Content Addressing**

IPFS CIDs derived from content hash. Modification changes CID.

**Merkle Proofs**

Reputation roots allow inclusion proofs. Prove record exists without revealing other records.

**Signature Chains**

Reputation records include evidence transaction hashes. Disputes can trace back to blockchain events.

## 6. SECURITY MODEL

### 6.1 Threat Model

**Adversary Capabilities**

- Can read all blockchain data
- Can submit transactions
- Can operate multiple identities
- Can attempt MEV extraction
- Can compromise individual keys
- Can attempt 51% attack on Layer 2

**Attack Vectors**

| Attack | Mitigation |
|---|---|
| Reentrancy | Checks-effects-interactions pattern |
| Front-running | Commit-reveal where applicable |
| Sybil | Minimum transaction count, weight by counterparty reputation |
| Oracle manipulation | Multi-source aggregation |
| Key compromise | Multi-sig for critical operations |
| Dispute manipulation | Evidence preservation, multi-jurisdiction support |

**Out of Scope**

- Quantum cryptanalysis
- Social engineering
- Physical key extraction
- Regulatory changes

### 6.2 Cryptographic Primitives

| Purpose | Algorithm |
|---|---|
| Identity | Ed25519 |
| Hashing | SHA-256 |
| Merkle trees | Keccak-256 |
| Encryption | AES-256-GCM |
| Key derivation | HKDF-SHA256 |

**Ed25519**

- Signature size: 64 bytes
- Public key size: 32 bytes
- Deterministic signatures (no nonce reuse vulnerability)
- Fast verification

**Commit-Reveal**

```
commitment = hash(value || nonce)
// Later:
verify(commitment, value, nonce)
```

### 6.3 Key Management

**Identity Keys**

- Generated locally from secure random
- Private key never transmitted
- Backup: encrypted export with passphrase

**Escrow Keys**

- Multi-sig: 2-of-3 (buyer, seller, platform)
- Platform key: hardware security module
- Key ceremony: documented, witnessed

**Upgrade Keys**

- Multi-sig: 3-of-5
- Timelock: 48 hours minimum
- Public announcement before execution

## 7. ECONOMIC MODEL

### 7.1 Fee Structure

| Transaction | Fee |
|---|---|
| Fixed price | 2.5% |
| Escrow | 2.5% |
| Auction | 2.5% |
| Dispute resolution | 5% |

**Distribution**

- Protocol treasury: 1.5%
- Reputation aggregators: 0.5%
- Dispute resolvers: 0.5%

### 7.2 Escrow Economics

**Gas Costs**

| Operation | Cost |
|---|---|
| Create escrow | $0.05–$0.20 |
| Release | $0.02–$0.10 |
| Dispute | $0.10–$0.50 |

**Minimum Viable Transaction**

Escrow becomes economically viable when transaction value exceeds approximately $10.

### 7.3 Incentive Alignment

**Buyers**

- Escrow protection
- Dispute resolution
- Reputation accumulation

**Sellers**

- Escrow guarantee
- Reputation accumulation
- Dispute resolution

**Aggregators**

- Fee share
- Reputation for accurate aggregation

**Protocol**

- Sustainable fee revenue
- Ecosystem growth

## 8. LEGAL FRAMEWORK

### 8.1 Jurisdiction

**Platform Position**

- Software provider, not marketplace operator
- Does not custody user funds
- Does not mediate disputes
- Does not verify identities

**Implications**

- Platform liability limited
- Users responsible for own compliance
- Dispute resolution optional
- Evidence preservation supports any jurisdiction

### 8.2 Dispute Resolution

**Evidence Types**

- Transaction hashes
- Signature verifications
- Timestamps
- Communication logs

**Resolution Options**

- Bilateral negotiation
- Arbitration
- Court proceedings

### 8.3 Evidence Preservation

**Retention**

- Blockchain: permanent
- IPFS: pinned while relevant
- Off-chain: 7 years recommended

**Integrity**

- Hashes prove content unchanged
- Signatures prove authorship
- Merkle proofs prove inclusion

## 9. IMPLEMENTATION

### 9.1 Technology Stack

| Component | Technology |
|---|---|
| Runtime | Bun 1.0+ |
| Language | TypeScript |
| Framework | Custom event handlers |
| Database | PostgreSQL |
| Cache | Redis |
| Streams | Redis Streams |
| Blockchain | Ethereum Optimism |
| Contracts | Hardhat, OpenZeppelin |
| Storage | IPFS |

### 9.2 Deployment Architecture

```
Load Balancer
  → Gateway Instance 1
  → Gateway Instance 2
  → Gateway Instance 3

Indexer (Single Instance)

Reputation Aggregator (Single Instance)

PostgreSQL (Primary + Replicas)

Redis (Cluster)

IPFS Node
```

### 9.3 Monitoring

**Metrics**

- Transaction volume
- Escrow state distribution
- Dispute rate
- Indexer lag
- Error rate

**Alerts**

- Indexer lag > 100 blocks
- Error rate > 1%
- Dispute spike
- Contract anomaly

## 10. ROADMAP

### 10.1 Phase 1: Foundation (Q1 2026)

- Core contracts deployed
- Basic indexer operational
- Fixed price trading
- Escrow trading
- Ed25519 reputation records
- Merkle tree aggregation

### 10.2 Phase 2: Ecosystem (Q2 2026)

- Auction contracts
- Multi-aggregator support
- Dispute resolution
- Mobile wallet integration
- Developer documentation

### 10.3 Phase 3: Governance (Q3 2026)

- Protocol governance
- Fee adjustment
- Aggregator selection
- Upgrade proposals

### 10.4 Phase 4: Decentralization (Q4 2026)

- Decentralized indexer
- Decentralized aggregation
- Cross-chain support
- Protocol autonomy

## 11. CONCLUSION

### 11.1 Summary

KNRT Marketplace implements a peer-to-peer NFT marketplace with:

- Zero-custody escrow
- Cryptographic reputation
- Hybrid storage
- Event-driven architecture
- Legal framework integration

### 11.2 Vision

Commerce infrastructure should:

- Minimize trust requirements
- Preserve user ownership
- Support jurisdiction diversity
- Enable composability

### 11.3 Call to Action

**Developers**

- Review contracts
- Run indexer
- Integrate API

**Security Researchers**

- Audit contracts
- Review cryptography
- Test edge cases

**Legal Professionals**

- Review jurisdiction handling
- Assess evidence preservation
- Provide feedback

## 12. APPENDIX

### 12.1 Glossary

| Term | Definition |
|---|---|
| Escrow | Third party holds funds until conditions met |
| Multi-sig | Multiple signatures required |
| Merkle tree | Hash tree enabling inclusion proofs |
| Commit-reveal | Two-phase protocol hiding intent |
| Sybil | Fake identities |
| Front-running | Exploiting transaction ordering |
| Reentrancy | Recursive contract exploit |
| Layer 2 | Scaling solution inheriting Ethereum security |
| Optimism | Specific Layer 2 implementation |
| Ed25519 | Elliptic curve signature algorithm |
| IPFS | Content-addressed storage network |
| CID | Content identifier |
| Indexer | Service processing blockchain events |
| Aggregator | Service computing reputation scores |
| Gateway | Service handling user requests |

### 12.2 References

**Standards**

- ERC-721: Non-fungible token standard
- ERC-1155: Multi-token standard
- EIP-712: Typed structured data hashing and signing
- RFC 8032: Edwards-curve Digital Signature Algorithm (Ed25519)

**Documentation**

- Optimism: https://docs.optimism.io
- OpenZeppelin: https://docs.openzeppelin.com
- IPFS: https://docs.ipfs.tech
- Merkle Proofs: https://en.wikipedia.org/wiki/Merkle_tree

**Research**

- Ethereum Foundation Blog
- Vitalik Buterin's blog
- Various DeFi security analyses

### 12.3 Contact

Author: Kaan Nurettin Ramazan
Contact: kaan@knrt.io
GitHub: kaan-knrt

**Contributions Welcome**

- Technical review
- Security analysis
- Implementation feedback
- Documentation improvements

**Feedback Channels**

- GitHub Issues
- Pull Requests
- Direct Contact

**Welcome to the Future of Digital Ownership**

Commerce infrastructure should minimize trust requirements, preserve user ownership, support jurisdiction diversity, and enable composability. This whitepaper describes one implementation of these principles.

© 2026 Kaan Nurettin Ramazan
This document is provided for informational purposes only.
No warranty is provided regarding accuracy or completeness.
Implementation details may differ from the reference architecture.
Technical specifications are subject to change without notice.
Nothing in this document constitutes legal, financial, or investment advice.
Consult appropriate professionals before implementation.
Document Version 1.0 | Date: 2026-01-15